1, Transaction properties:-

In DBMS, a transaction must satisfy four key properties known as ACID properties

1. Atomicity (All or nothing):-

A transaction is treated as a single unit

- Either all operations are completed or none are applied
- If any part fails → the entire transaction is rolled back

Ex:- Debit from A ✓

Credit from B ✗

→ entire transaction is undone

2) Consistency (valid state)

A transaction must take the database from one valid state to other.

Ex:- If total balance before transaction = ₹10,000

After transaction → must still be ₹10,000

3) Isolation (no interface)

Multiple transaction can run simultaneously, but they shouldn't effect each other

Ex:- Two users booking the same seat.

→ System ensures only one succeeds

4) Durability :- (permanent changes)

once a transaction is successfully completed, changes are permanent.

Ex :- After money transfer confirmation, data remains saved even after power failure.

2) Concurrency control Importance.

Concurrency means multiple transactions executed at the same time in a data base system.

purpose : Improve performance
Increase resource utilization
Support multiuser environment

Issue :- Lost update
dirty read
Inconsistent data

It is the reason for DBMS user Concurrency control (Locking, time stamps)

Two types of Concurrency processing :

1. Interleved processing :-
Execution of multiple transactions by alternating their operations in a single CPU

Ex :- T_1 : read → Write
T_2 : read → Write

Execution order : T_1 (read) → T_2 (read) → T_1 (write) → T_2 (write)

2, Parallel processing :-

Execution of multiple transactions simultaneously using multiple CPU's / processors

Ex :- T_1 runs on CPU1
T_2 runs on CPU2

3) Serial & non serial schedule with example

Serial solution :- A serial schedule is one in which transaction are executed one after another without any interleaving one transaction completes fully before the next begins, so it always produces correct results but is less different.

Non serial solution :-

A non serial schedule is one in which operation of multiple transaction are interleaved. It improves performance but may cause inconsistency if not properly controlled.

Examples :- Non serial schedule

T_1	T_2
R(A)	
W(A)	
	R(A)
	W(A)
R(A)	
W(A)	

Serial Schedule

T_1	T_2
R(A)	
W(A)	
R(A)	
W(A)	
	R(A)
	W(A)
	R(A)
	W(A)

4) Serializability with example.

The serializability of schedule is used to find non-serial schedule that allow the transaction to execute concurrently without interfering with one another.

⇒ A serializable schedule is non serial schedule that produces the same result as a serial schedule. even through transaction run concurrency to find result remains correct.

⇒ There are two types of serializability:

* Conflict serializability
* View serializability

⇒ If guarantees that even multiuser environments, the database remains consistent & reliable

Ex:- Serializable with different data items

Transactions:

T_1 : R(A), W(A)

T_2 : R(B), W(B)

Schedule : $R_1(A) \rightarrow R_2(B) \rightarrow W_1(A) \rightarrow W_2(B)$

- No conflicts (different data items A & B)
- Can be rearranged in any order

∴ Equivalent serial order : $T_1 \rightarrow T_2$ (or) $T_2 \rightarrow T_1$

∴ Serializable

5) Conflict Serializability & equivalent with examples

Conflict Serializability :- It is a type of serializability where a schedule can be transformed into serial schedule by swapping non-conflicting operations.

Two operations are said to be in conflict if :-

⇒ They belong to different transactions

⇒ They operate on same data item

⇒ Atleast one operation is a write.

Ex :- List all conflicting operations & determine dependency between transactions.

T_1	T_2	T_3	T_4
	R(x)		
		W(x)	
		Commit	
W(x)			
Commit			
	W(y)		
	R(z)		
	Commit		
			R(x)
			R(y)
			Commit

$R_2(x), W_3(x)$ $(T_2 \rightarrow T_3)$

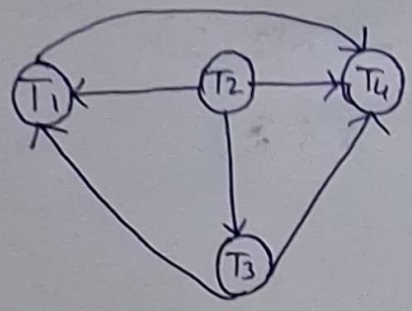
$R_2(x), W_1(x)$ $(T_2 \rightarrow T_1)$

$W_3(x), W_1(x)$ $(T_3 \rightarrow T_1)$

$W_3(x), R_4(x)$ $(T_3 \rightarrow T_4)$

$W_1(x), R_4(x)$ $(T_1 \rightarrow T_4)$

$W_2(Y), R_4(Y)$ $(T_2 \rightarrow T_4)$



clearly there exists no cycle in precedence graph

∴ given schedule 's' is Conflict Serializable

Conflict Equivalent:- Two schedules are said to be Conflict equivalent when one can be transformed to another by swapping non-Conflict operations

Ex:-

T_1	T_2
R(A)	
W(A)	
R(B)	
	R(A)
	W(A)

schedules

T_1	T_2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	

schedule s'

R(A), R(A)
R(B) R(A)
R(B) W(A)
W(B) R(A)
W(A) W(B)
R(A) W(A)

} non Conflict pairs

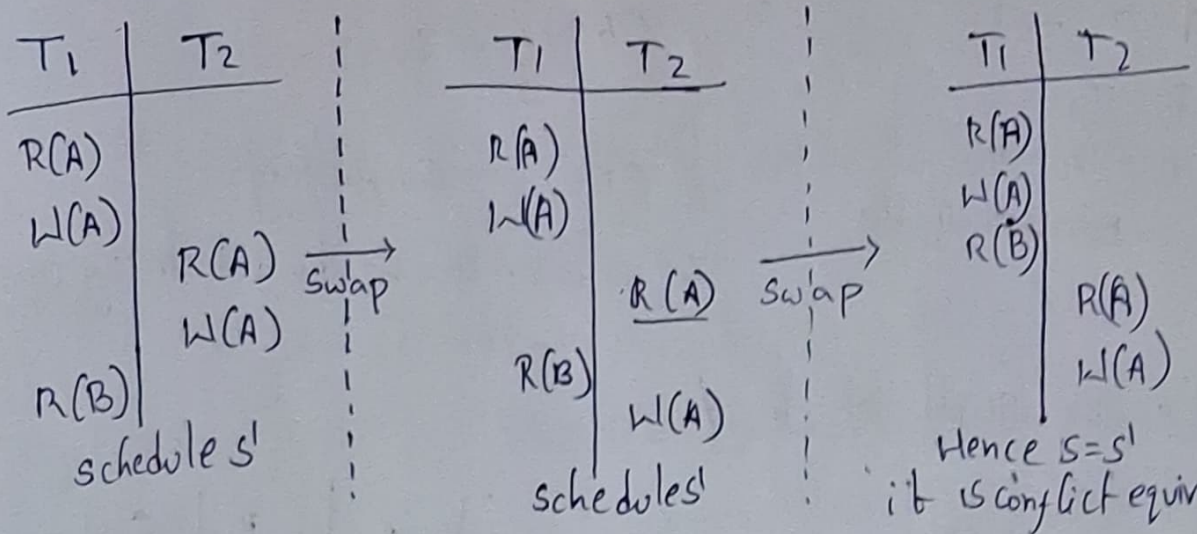
R(A) W(A)
W(A) W(A)
W(A) R(A)

} Conflict pairs

step 1:- check adjacent non conflict pairs as per rules

step 2:- If swap it

Consider from test:



6) View serializable & equivalent with examples:

View serializability :- A schedule is correct if it gives the same result as some serial (one by one) execution of transactions

3 Conditions:-

1. check initial read
   → If transaction reads original value, it should remain same
2. read from (dependency)
   → If T_2 reads value written by T_1, it must remain same
3. Final write
   → Last transaction writing a data item must be same

i) check for conflict serializability

ii) check for view serializability

T_1	T_2	T_3
R(A)	R(A)	R(B)
W(A)	R(C)	
	R(B)	
	W(B)	W(C)

	A	B	C
Initial read	T_1, T_2	T_3, T_2	T_2
update	T_1	T_2	T_3
Final update	T_1	T_2	T_3

check item by item

A : $T_2 \rightarrow T_1$

B : $T_3 \rightarrow T_2$

C : $T_2 \rightarrow T_3$

∴ non serializable

(A B) } $\rightarrow T_3 \rightarrow T_2 \rightarrow T_1$

(B C) } $\rightarrow$ Conflicting

So no order can be divided strictly one basis of vs.

We can simply check it from vs :-

T_1	T_2	T_3
R(A)		
	R(A)	R(B)
W(A)		
	R(C)	
	R(B)	
W(C)	W(B)	

	A	B	C
Initial read	T_1, T_2	T_3, T_2	T_2
update	T_1	T_2	T_1
Final update	T_1	T_2	T_1

A : $T_2 \rightarrow T_1$

B : $T_3 \rightarrow T_2$

C : $T_2 \rightarrow T_1$

$T_3 : T_2 : T_1$

(A B) } $\rightarrow T_3 \rightarrow T_2 \rightarrow T_1$

(B C) } $\rightarrow T_3 \rightarrow T_2 \rightarrow T_1$

(A C) } $\rightarrow T_2 \rightarrow T_1$

∴ view serializable

View equivalent schedule :-

S1 :

T_1	T_2	T_3
R(A)		
	W(A)	
W(A)		
		W(A)

Initial read : by T_1 on A

Final write : by T_3 on A

W|R Conflict : not present

S2 :

T_1	T_2	T_3
R(A)		
W(A)		
	W(A)	
		W(A)

Initial read : by T_1 on A

Final write : by T_3 on A

W|R Conflict : not present

So S1 & S2 are view equivalent schedule

movable Schedule :-

A schedule is recoverable if:

* Transaction T_j commits only after T_i commits
* If T_j has read data written by T_i

$$Commit(T_i) \rightarrow Commit(T_j)$$

A recoverable schedule in DBMS ensures transaction commits only after transaction if it depends on have committed

3 types of removable schedules :-

1. Cascading Schedule :- It is a schedule which a transaction reads uncommitted data from another transaction If first transaction fails, then all dependent transactions must roll back

T_1	T_2	T_3	T_4
R(A)			
W(A)			
	R(A)		
	W(A)		
		R(A)	
		W(A)	
			R(A)
			W(A)

Failure

* Failure of transactions T_1, T_2, T_3 causes the transaction T_2, T_3, T_4 to roll back respectively

∴ such roll back is called Cascading Rollback

ii) Cascadeless schedule: It is a schedule in which a transaction reads data only after the transaction that wrote it has committed.

T_1	T_2	T_3
R(A)		
W(A)		
Commit		
	R(A)	
	W(A)	
	Commit	
		R(A)
		W(A)
		Commit

Cascadeless schedule allows only commited read operations →

It allows uncommited write operations

T_1	T_2
R(A)	
W(A)	
	W(A)
	|| uncom
Commit	

iii) Strict Schedule:-

A schedule is strict if no transaction can read or write a data item until transaction that last wrote it commits or aborts

T_1	T_2
W(A)	
Commit / rollback	
	R(A)/W(A)